

Priority Queues

Comp Sci 214, Spring 2025

Don't forget to check in on Youhere!

Beyond FIFO and LIFO

Today: short break from graph algorithms;
we'll get back to those soon

Recall the two ADTs we saw to keep track of to-do lists:

- *Stacks*, elements come out last-in first-out (LIFO)
- *Queues*, elements come out first-in first-out (FIFO)

Beyond FIFO and LIFO

Today: short break from graph algorithms;
we'll get back to those soon

Recall the two ADTs we saw to keep track of to-do lists:

- *Stacks*, elements come out last-in first-out (LIFO)
- *Queues*, elements come out first-in first-out (FIFO)

But sometimes, the order in which we want to process elements is unrelated to the order they came in!

- E.g., emergency room: most serious injuries first

New ADT, *priority queue*: first in, **most urgent** out

- Many of you invented it in your hw 2 self-eval!

Priority Queue ADT

Looks like: $\langle 2:g, 5:i, 5:b, 17:c, 89:q \rangle$ (note sorting)

```
interface PRIORITY_QUEUE[T]:  
  def empty?(self) -> bool?  
  def insert(self, priority: num?, value: T) -> NoneC  
  def remove_min(self) -> T
```

Behavior: `remove_min` finds, removes pair with *lowest* priority

- And returns the value
- Lowest priority = highest *urgency*; needs to come out first
 - ▶ Confusing, but that's the terminology everyone uses
 - ▶ **Think:** my #1 priority is more urgent than my #2 priority
- Multiple elements can have the same priority, no problem

Variants:

- Separate `find_min` and `remove_min`
- Combined priorities and elements
- → see textbook and homework 5

Priority Queue Examples

Emergency Room Triage:

Level I	Resuscitation	see patient immediately
Level II	Emergency	within 15 minutes
Level III	Urgency	within 30 minutes
Level IV	Less Urgency	within 60 minutes
Level V	Non Urgency	within 120 minutes

$pq = \langle 3: Rod, 5: Amy, 5: Shiv \rangle_{<}$

Priority Queue Examples

Emergency Room Triage:

Level I	Resuscitation	see patient immediately
Level II	Emergency	within 15 minutes
Level III	Urgency	within 30 minutes
Level IV	Less Urgency	within 60 minutes
Level V	Non Urgency	within 120 minutes

`pq =` $\langle 3: Rod, 5: Amy, 5: Shiv \langle$

`pq.insert(1, 'Brianna')`

$\rightarrow \langle 1: Brianna, 3: Rod, 5: Amy, 5: Shiv \langle$

Priority Queue Examples

Emergency Room Triage:

Level I	Resuscitation	see patient immediately
Level II	Emergency	within 15 minutes
Level III	Urgency	within 30 minutes
Level IV	Less Urgency	within 60 minutes
Level V	Non Urgency	within 120 minutes

`pq =` $\langle 3: Rod, 5: Amy, 5: Shiv \langle$

`pq.insert(1, 'Brianna')`

$\rightarrow \langle 1: Brianna, 3: Rod, 5: Amy, 5: Shiv \langle$

`pq.remove_min()` $\rightarrow$ Brianna

Priority Queue Examples

Managing homework deadlines:

⟨ 2/10: physics, 2/14: 214, 2/28: calculus, 3/2: english ⟩_{earlier}

pq.remove_min() → ???

Priority Queue Examples

Managing homework deadlines:

`< 2/10: physics, 2/14: 214, 2/28: calculus, 3/2: english <_earlier`

`pq.remove_min()` $\rightarrow$ physics

Priorities don't have to be numbers!

Anything comparable is fair game!

(Cf. sorting lecture and custom comparators.)

Whoever creates the PQ needs to provide a proper comparator.

Priority Queue Laws

$$\{\} \quad \boxed{\langle \langle < } .empty?() \Rightarrow \text{True} \quad \{\}$$

$$\{\forall i. p_i \leq p_{i+1}\} \quad \boxed{\langle p_0 : e_0, \dots, p_n : e_n \langle < } .empty?() \Rightarrow \text{False} \quad \{\}$$

$$\left\{ q = \boxed{\langle p_0 : e_0, \dots, p_{k-1} : e_{k-1}, p_{k+1} : e_{k+1}, p_n : e_n \langle < } \wedge \forall i. p_i \leq p_{i+1} \right\}$$

$$q.insert(p_k, e_k) \Rightarrow \text{None}$$

$$\left\{ q = \boxed{\langle p_0 : e_0, \dots, p_{k-1} : e_{k-1}, p_k : e_k, p_{k+1} : e_{k+1}, p_n : e_n \langle < } \right\}$$

$$\left\{ q = \boxed{\langle p_0 : e_0, p_1 : e_1, \dots, p_n : e_n \langle < } \wedge \forall i. p_i \leq p_{i+1} \right\}$$

$$q.remove_min() \Rightarrow e_0$$

$$\left\{ q = \boxed{\langle p_1 : e_1, \dots, p_n : e_n \langle < } \right\}$$

$\leq \rightarrow$ in case of ties, any of the tied elements is fine!

Stop! Question time!

Before we move on to building priority queues...

Anything unclear so far?

Anything I missed?

Some candidate priority queues

A (min-)priority queue provides these operations:

- insert: adds an element
- remove_min: removes the element with the lowest priority

Any suggestions for how we could implement one using the tools we've seen earlier in the class?

Some candidate priority queues

A (min-)priority queue provides these operations:

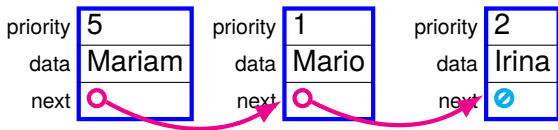
- insert: adds an element
- remove_min: removes the element with the lowest priority

Any suggestions for how we could implement one using the tools we've seen earlier in the class?

	insert	remove_min
unsorted list	$\mathcal{O}(1)$	$\mathcal{O}(n)$
sorted list	$\mathcal{O}(n)$	$\mathcal{O}(1)$

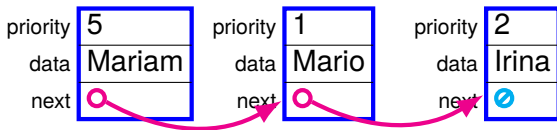
(n is the number of elements in the priority queue.)

Unsorted list priority queue



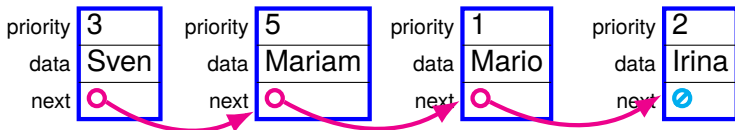
- insert: add a node at the start: $\mathcal{O}(1)$

Unsorted list priority queue



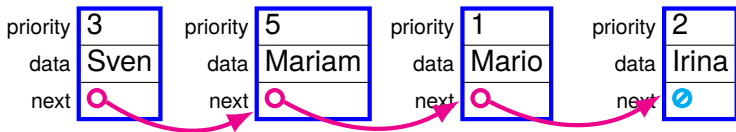
- insert: add a node at the start: $\mathcal{O}(1)$
 - ▶ Inserting Sven with priority 3

Unsorted list priority queue



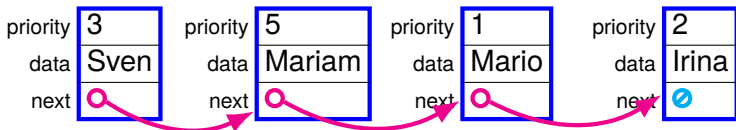
- insert: add a node at the start: $\mathcal{O}(1)$
 - ▶ Inserting Sven with priority 3

Unsorted list priority queue



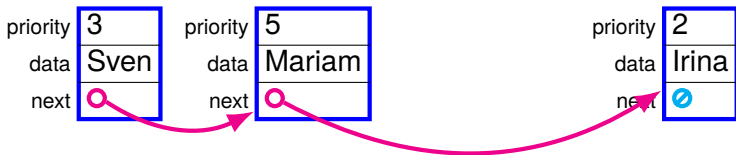
- insert: add a node at the start: $\mathcal{O}(1)$
 - ▶ Inserting Sven with priority 3
- remove_min: linear search for minimum priority: $\mathcal{O}(n)$
 - ▶ Have to look through entire list, just in case!

Unsorted list priority queue



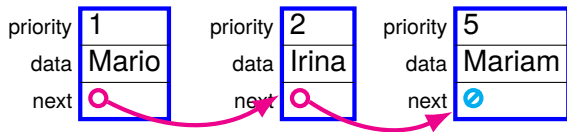
- insert: add a node at the start: $\mathcal{O}(1)$
 - ▶ Inserting Sven with priority 3
- remove_min: linear search for minimum priority: $\mathcal{O}(n)$
 - ▶ Have to look through entire list, just in case!
 - ▶ Mario is lowest priority

Unsorted list priority queue



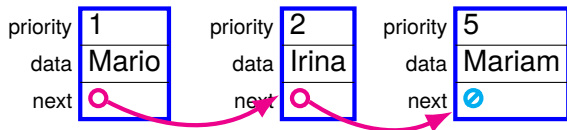
- insert: add a node at the start: $\mathcal{O}(1)$
 - ▶ Inserting Sven with priority 3
- remove_min: linear search for minimum priority: $\mathcal{O}(n)$
 - ▶ Have to look through entire list, just in case!
 - ▶ Mario is lowest priority

Sorted list priority queue



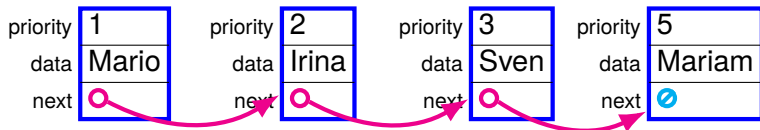
- insert: find the right position and add a node: $\mathcal{O}(n)$

Sorted list priority queue



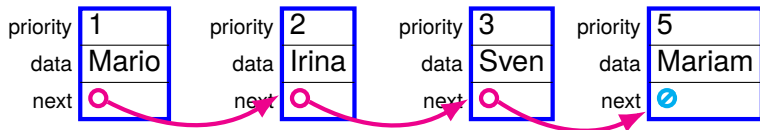
- insert: find the right position and add a node: $\mathcal{O}(n)$
 - ▶ Inserting Sven with priority 3

Sorted list priority queue



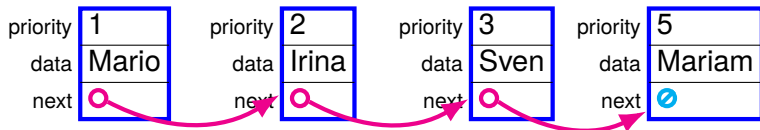
- insert: find the right position and add a node: $\mathcal{O}(n)$
 - ▶ Inserting Sven with priority 3

Sorted list priority queue



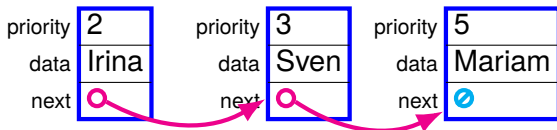
- insert: find the right position and add a node: $\mathcal{O}(n)$
 - ▶ Inserting Sven with priority 3
- remove_min: remove first element: $\mathcal{O}(1)$

Sorted list priority queue



- insert: find the right position and add a node: $\mathcal{O}(n)$
 - ▶ Inserting Sven with priority 3
- remove_min: remove first element: $\mathcal{O}(1)$
 - ▶ Mario is lowest priority

Sorted list priority queue



- insert: find the right position and add a node: $\mathcal{O}(n)$
 - ▶ Inserting Sven with priority 3
- remove_min: remove first element: $\mathcal{O}(1)$
 - ▶ Mario is lowest priority

Introducing the binary heap

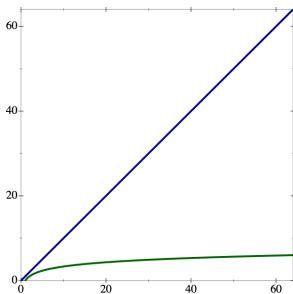
So far, we have to pick our poison: which operation is $\mathcal{O}(n)$?
Can we get the best of both worlds?

Introducing the binary heap

So far, we have to pick our poison: which operation is $\mathcal{O}(n)$?
Can we get the best of both worlds?

Almost!

We can't get $\mathcal{O}(1)$ for both operations,* but we can get $\mathcal{O}(\log n)$!
(Which is almost as good.)



The solution: the *binary heap* data structure.

Introducing the binary heap

So far, we have to pick our poison: which operation is $\mathcal{O}(n)$?
Can we get the best of both worlds?

Almost!

We can't get $\mathcal{O}(1)$ for both operations,* but we can get $\mathcal{O}(\log n)$!
(Which is almost as good.)

* We can get $\mathcal{O}(1)$ and $\mathcal{O}(\log n)$, but with some caveats, and with a much more complicated data structure, the *Fibonacci heap*.
Well beyond the scope of this class.

The solution: the *binary heap* data structure.

Introducing the binary heap

A *binary heap* is a *complete* binary tree that is *heap-ordered*.

Introducing the binary heap

A *binary heap* is a *complete* binary tree that is *heap-ordered*.

A k -ary tree is *complete* if every level is full of nodes. The last level can have nodes missing, but must be filled left-to-right.

Introducing the binary heap

A *binary heap* is a *complete* binary tree that is *heap-ordered*.

A k -ary tree is *complete* if every level is full of nodes. The last level can have nodes missing, but must be filled left-to-right.

A tree is *heap-ordered* if each element is less than or equal to its children.

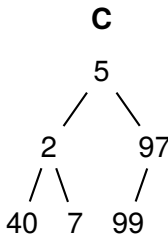
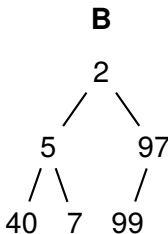
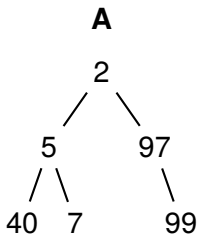
Introducing the binary heap

A *binary heap* is a *complete* binary tree that is *heap-ordered*.

A k -ary tree is *complete* if every level is full of nodes. The last level can have nodes missing, but must be filled left-to-right.

A tree is *heap-ordered* if each element is less than or equal to its children.

QUIZ: Which of these is a binary heap?



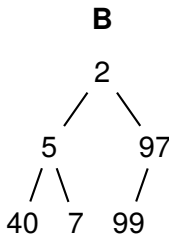
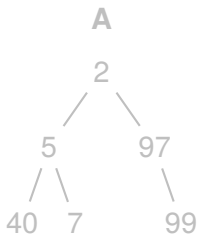
Introducing the binary heap

A *binary heap* is a *complete* binary tree that is *heap-ordered*.

A k -ary tree is *complete* if every level is full of nodes. The last level can have nodes missing, but must be filled left-to-right.

A tree is *heap-ordered* if each element is less than or equal to its children.

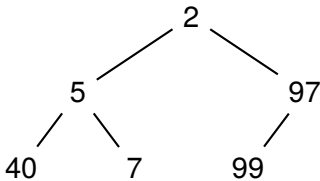
QUIZ: Which of these is a binary heap?



Answer: B. A is not complete. C is not heap-ordered.

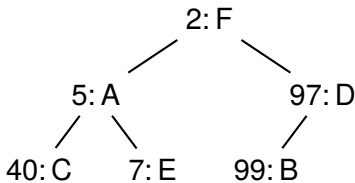
I am not a number, I am a free man!

But wait! I thought we wanted to keep track of *elements* along with their priorities. But all I see in these heaps is numbers!



I am not a number, I am a free man!

But wait! I thought we wanted to keep track of *elements* along with their priorities. But all I see in these heaps is numbers!



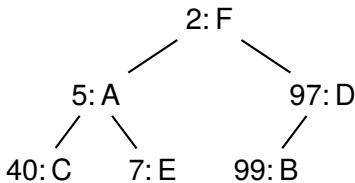
One can imagine a “payload” (e.g., patient, hw) at each node.

But since we only need the priorities to determine the shape, we’ll only show priorities and assume payloads follow along.

Keeps things simple. Standard in the data structures world.

I am not a number, I am a free man!

But wait! I thought we wanted to keep track of *elements* along with their priorities. But all I see in these heaps is numbers!



But when implementing, need to think of payloads too!

Either store priorities and payloads separately.

OR use structs with two fields as your elements

AND a custom comparator that only looks at priority!

```
lambda x, y: x.priority < y.priority
```

Stop! Question time!

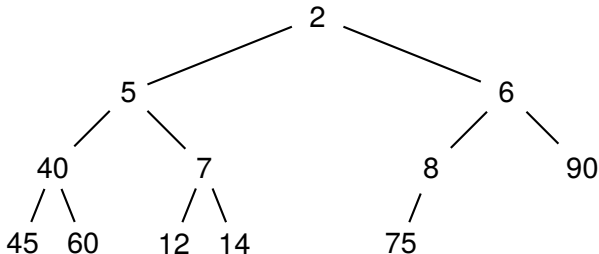
Before we move on to binary heap operations...

Anything unclear so far?

Anything I missed?

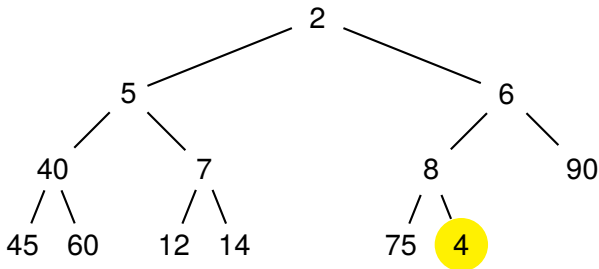
Binary heap insertion

1. Add the new element at the end, breaking the invariant
 - The end is the only place we can add to and stay complete
2. “Bubble up” to restore invariant



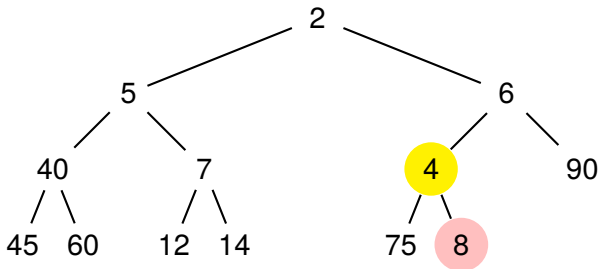
Binary heap insertion

1. Add the new element at the end, breaking the invariant
 - The end is the only place we can add to and stay complete
2. “Bubble up” to restore invariant



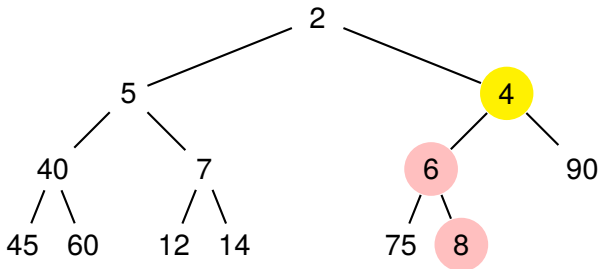
Binary heap insertion

1. Add the new element at the end, breaking the invariant
 - The end is the only place we can add to and stay complete
2. “Bubble up” to restore invariant



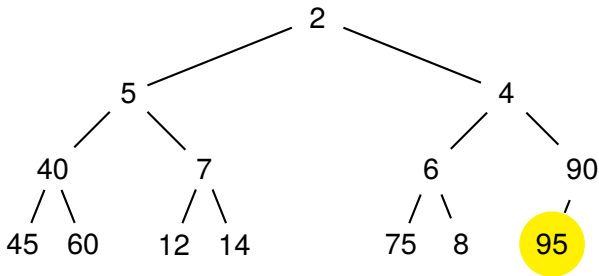
Binary heap insertion

1. Add the new element at the end, breaking the invariant
 - The end is the only place we can add to and stay complete
2. “Bubble up” to restore invariant



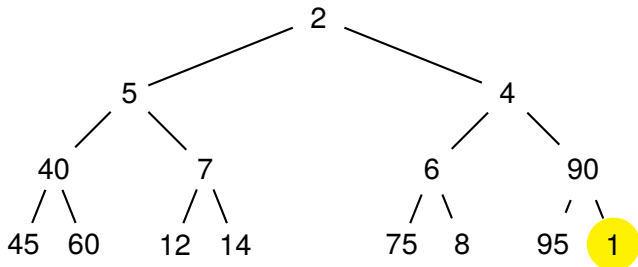
Binary heap insertion

1. Add the new element at the end, breaking the invariant
 - The end is the only place we can add to and stay complete
2. “Bubble up” to restore invariant



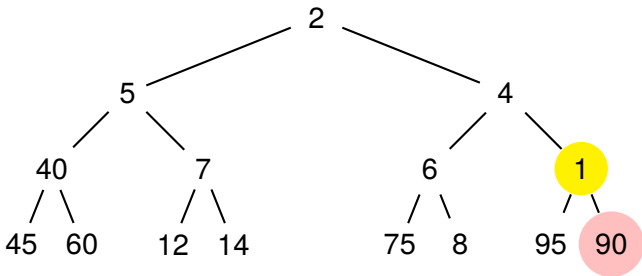
Binary heap insertion

1. Add the new element at the end, breaking the invariant
 - The end is the only place we can add to and stay complete
2. “Bubble up” to restore invariant



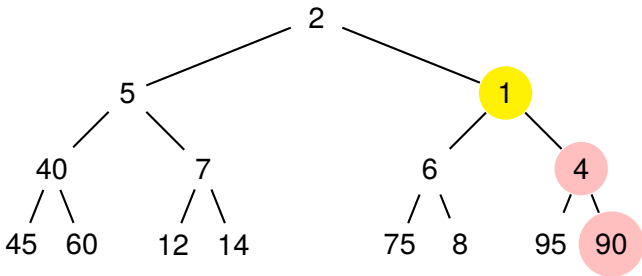
Binary heap insertion

1. Add the new element at the end, breaking the invariant
 - The end is the only place we can add to and stay complete
2. “Bubble up” to restore invariant



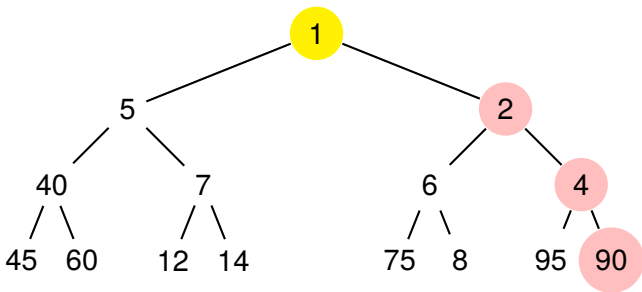
Binary heap insertion

1. Add the new element at the end, breaking the invariant
 - The end is the only place we can add to and stay complete
2. “Bubble up” to restore invariant



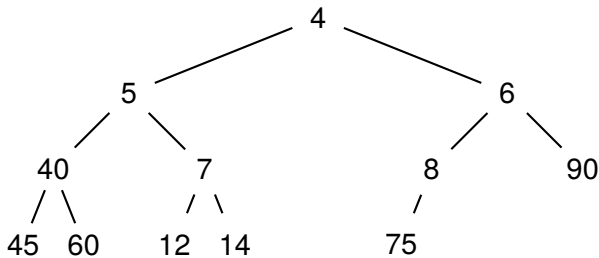
Binary heap insertion

1. Add the new element at the end, breaking the invariant
 - The end is the only place we can add to and stay complete
2. “Bubble up” to restore invariant



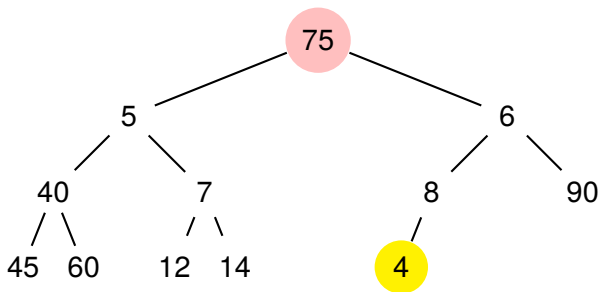
Binary heap remove-min

1. Replace the root with the last element of the heap
 - Root is min element!
 - Last is the only one we can remove and still be complete
2. “Percolate down” to restore invariant
 - Swap with smallest child; needs to come out on top



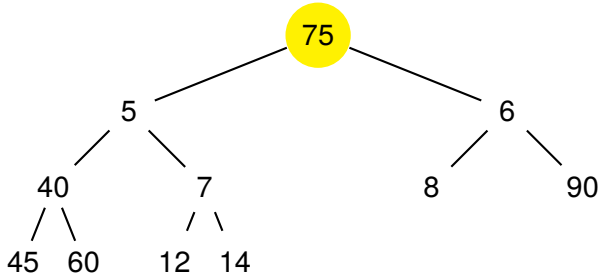
Binary heap remove-min

1. Replace the root with the last element of the heap
 - Root is min element!
 - Last is the only one we can remove and still be complete
2. "Percolate down" to restore invariant
 - Swap with smallest child; needs to come out on top



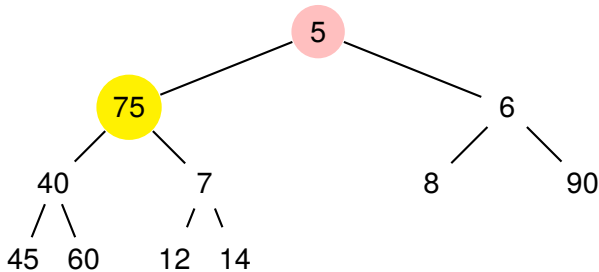
Binary heap remove-min

1. Replace the root with the last element of the heap
 - Root is min element!
 - Last is the only one we can remove and still be complete
2. "Percolate down" to restore invariant
 - Swap with smallest child; needs to come out on top



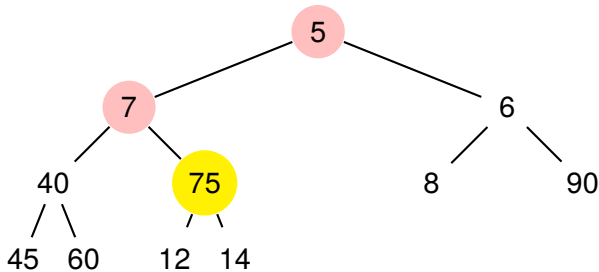
Binary heap remove-min

1. Replace the root with the last element of the heap
 - Root is min element!
 - Last is the only one we can remove and still be complete
2. “Percolate down” to restore invariant
 - Swap with smallest child; needs to come out on top



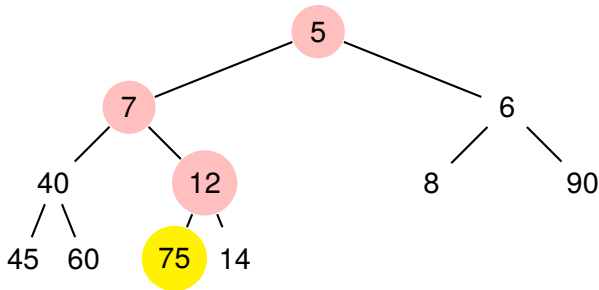
Binary heap remove-min

1. Replace the root with the last element of the heap
 - Root is min element!
 - Last is the only one we can remove and still be complete
2. “Percolate down” to restore invariant
 - Swap with smallest child; needs to come out on top



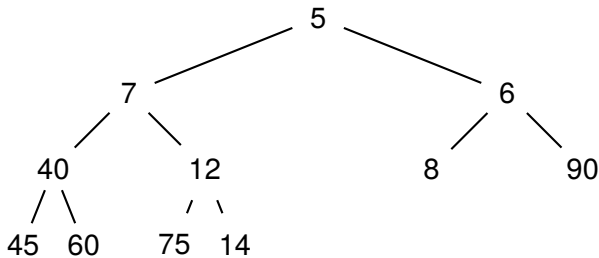
Binary heap remove-min

1. Replace the root with the last element of the heap
 - Root is min element!
 - Last is the only one we can remove and still be complete
2. “Percolate down” to restore invariant
 - Swap with smallest child; needs to come out on top



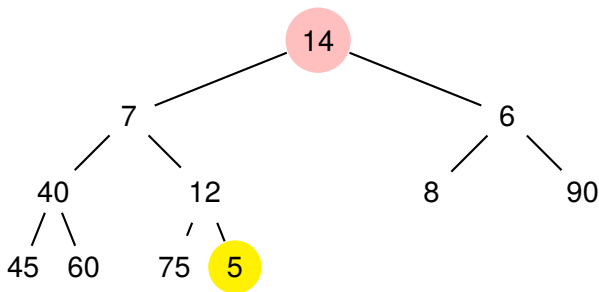
Binary heap remove-min

1. Replace the root with the last element of the heap
 - Root is min element!
 - Last is the only one we can remove and still be complete
2. “Percolate down” to restore invariant
 - Swap with smallest child; needs to come out on top



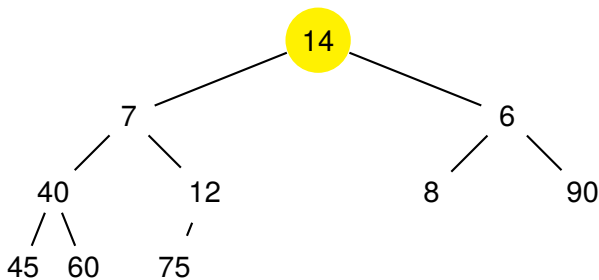
Binary heap remove-min

1. Replace the root with the last element of the heap
 - Root is min element!
 - Last is the only one we can remove and still be complete
2. "Percolate down" to restore invariant
 - Swap with smallest child; needs to come out on top



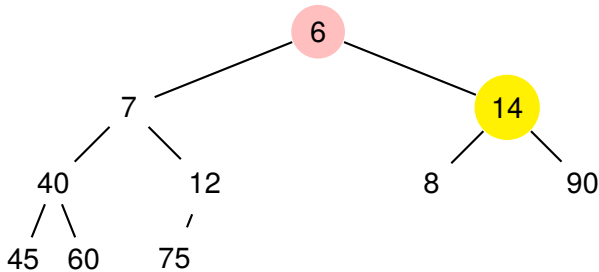
Binary heap remove-min

1. Replace the root with the last element of the heap
 - Root is min element!
 - Last is the only one we can remove and still be complete
2. "Percolate down" to restore invariant
 - Swap with smallest child; needs to come out on top



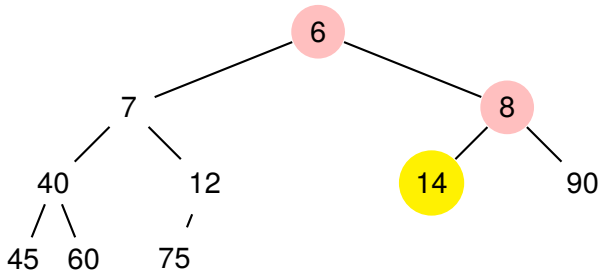
Binary heap remove-min

1. Replace the root with the last element of the heap
 - Root is min element!
 - Last is the only one we can remove and still be complete
2. “Percolate down” to restore invariant
 - Swap with smallest child; needs to come out on top



Binary heap remove-min

1. Replace the root with the last element of the heap
 - Root is min element!
 - Last is the only one we can remove and still be complete
2. “Percolate down” to restore invariant
 - Swap with smallest child; needs to come out on top



Binary heap operations time complexity

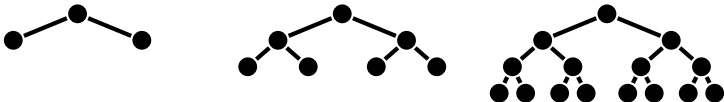
In the worst case, both inserting and removing make changes along a path between *one* leaf and the root.

How long is such a path? → *height* of the tree

Binary heap operations time complexity

In the worst case, both inserting and removing make changes along a path between *one* leaf and the root.

How long is such a path? $\rightarrow$ *height* of the tree



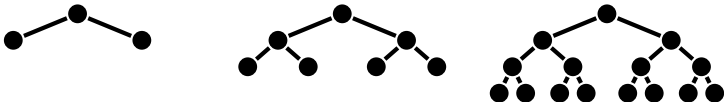
A tree of height ____		has at most ____ nodes	
2	3	$(= 2^2 - 1)$	
3	7	$(= 2^3 - 1)$	
4	15	$(= 2^4 - 1)$	
$\vdots$	$\vdots$		

$\rightarrow$ height is $\mathcal{O}(\log n) \rightarrow$ inserting/removing are both log time!

Binary heap operations time complexity

In the worst case, both inserting and removing make changes along a path between *one* leaf and the root.

How long is such a path? → *height* of the tree



Of course, this is not true for *any* tree with n nodes!

Only for *balanced* trees.

But *complete* trees are guaranteed to be balanced!

(Lots more on trees and balance later in the quarter.)

Binary heap operations time complexity

In the worst case, both inserting and removing make changes along a path between *one* leaf and the root.

How long is such a path? → *height* of the tree



Q: Where do logs come from?

Binary heap operations time complexity

In the worst case, both inserting and removing make changes along a path between *one* leaf and the root.

How long is such a path? → *height* of the tree



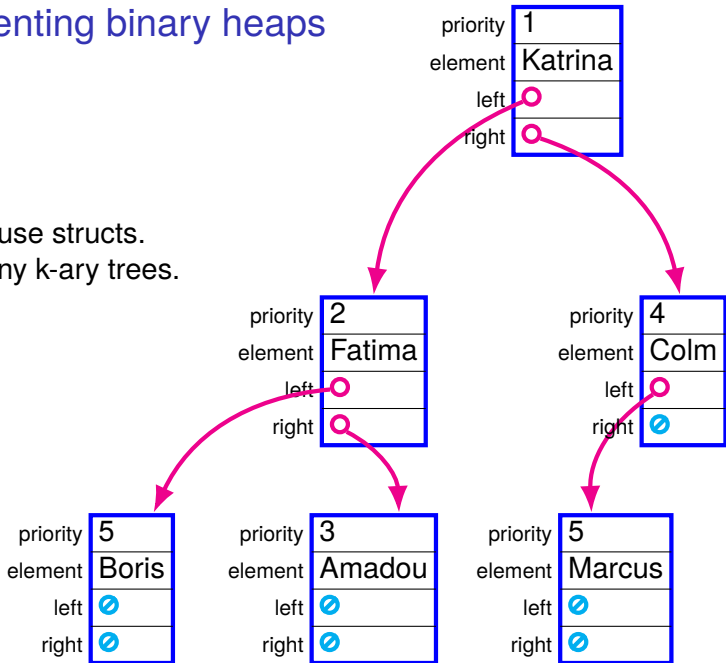
Q: Where do logs come from?

A: From trees!

(I'll be here all quarter.)

Representing binary heaps

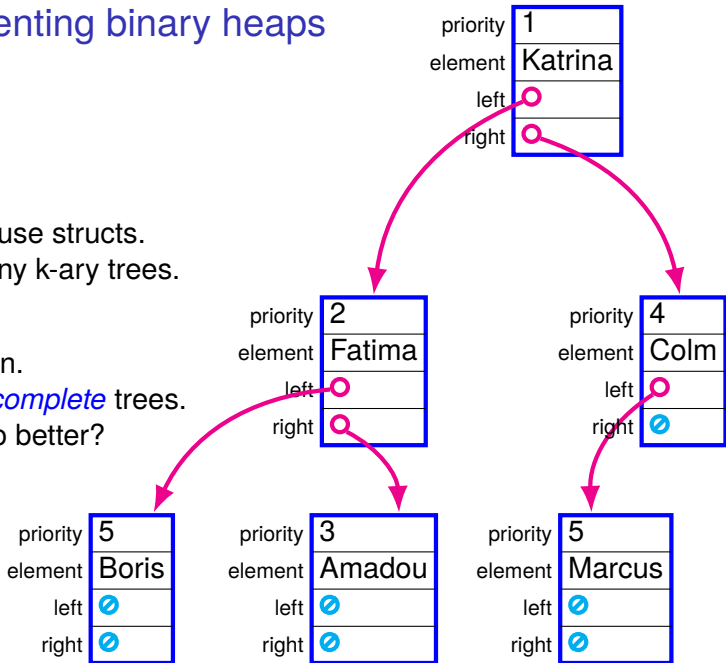
We could use structs.
Just like any k-ary trees.



Representing binary heaps

We could use structs.
Just like any k-ary trees.

But hold on.
We have *complete* trees.
Can we do better?



Representing complete trees

Let's do a magic trick.

- Everyone, take a sheet of paper and a pen

Representing complete trees

Let's do a magic trick.

- Everyone, take a sheet of paper and a pen
- Nico picks a number between 10 and 20

Representing complete trees

Let's do a magic trick.

- Everyone, take a sheet of paper and a pen
- Nico picks a number between 10 and 20
- Draw a complete binary tree with that many nodes

My claim: everyone will draw trees with the exact same shape!

Representing complete trees

Let's do a magic trick.

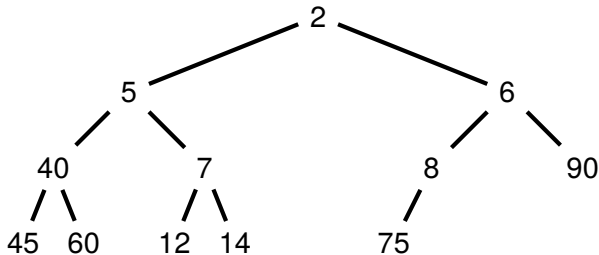
- Everyone, take a sheet of paper and a pen
- Nico picks a number between 10 and 20
- Draw a complete binary tree with that many nodes

My claim: everyone will draw trees with the exact same shape!

Here's the trick: there's only *one* possible shape for any complete binary (or k -ary) tree with n nodes!

The super cool thing about binary heaps

Instead of storing it as an actual tree with arrows, because the shape is determined solely by the number of nodes...



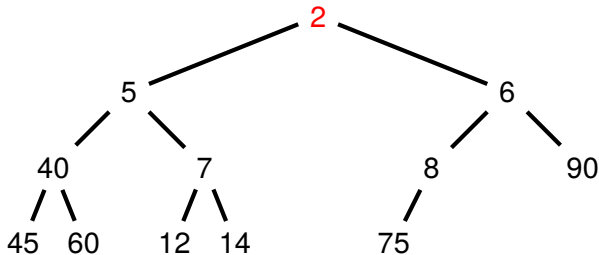
...we can store a binary heap in an array!

In order by level, from left to right within each level:

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19
2	5	6	40	7	8	90	45	60	12	14	75								

The super cool thing about binary heaps

Instead of storing it as an actual tree with arrows, because the shape is determined solely by the number of nodes...



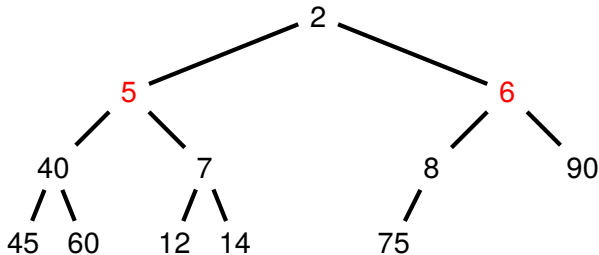
...we can store a binary heap in an array!

In order by level, from left to right within each level:

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19
2	5	6	40	7	8	90	45	60	12	14	75								

The super cool thing about binary heaps

Instead of storing it as an actual tree with arrows, because the shape is determined solely by the number of nodes...



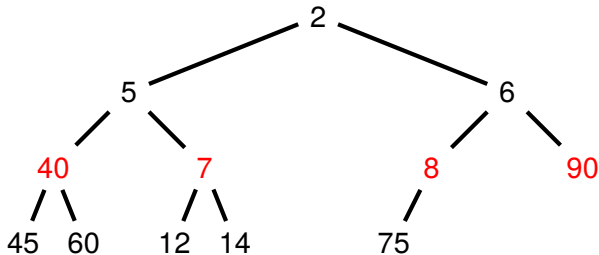
...we can store a binary heap in an array!

In order by level, from left to right within each level:

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19
2	5	6	40	7	8	90	45	60	12	14	75								

The super cool thing about binary heaps

Instead of storing it as an actual tree with arrows, because the shape is determined solely by the number of nodes...



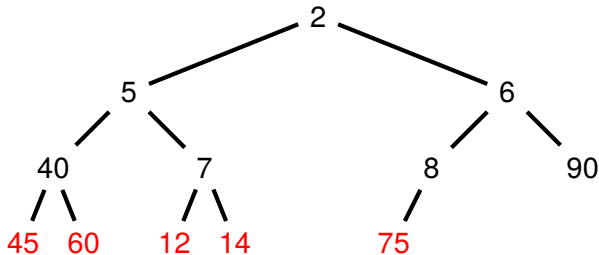
...we can store a binary heap in an array!

In order by level, from left to right within each level:

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19
2	5	6	40	7	8	90	45	60	12	14	75								

The super cool thing about binary heaps

Instead of storing it as an actual tree with arrows, because the shape is determined solely by the number of nodes...



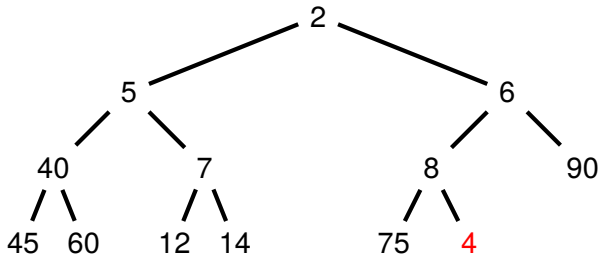
...we can store a binary heap in an array!

In order by level, from left to right within each level:

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19
2	5	6	40	7	8	90	45	60	12	14	75								

The super cool thing about binary heaps

Instead of storing it as an actual tree with arrows, because the shape is determined solely by the number of nodes...



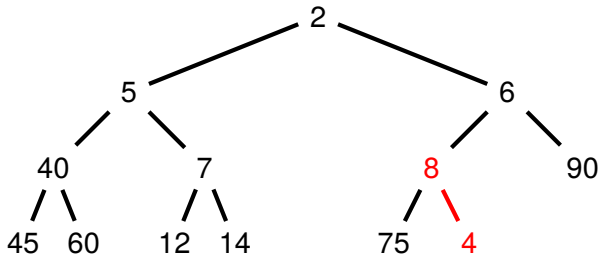
...we can store a binary heap in an array!

In order by level, from left to right within each level:

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19
2	5	6	40	7	8	90	45	60	12	14	75	4							

The super cool thing about binary heaps

Instead of storing it as an actual tree with arrows, because the shape is determined solely by the number of nodes...



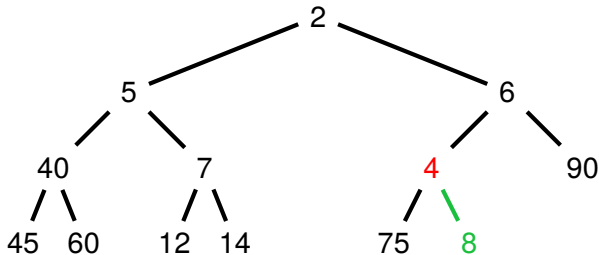
...we can store a binary heap in an array!

In order by level, from left to right within each level:

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19
2	5	6	40	7	8	90	45	60	12	14	75	4							

The super cool thing about binary heaps

Instead of storing it as an actual tree with arrows, because the shape is determined solely by the number of nodes...



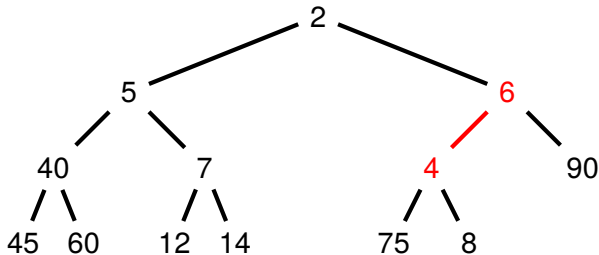
...we can store a binary heap in an array!

In order by level, from left to right within each level:

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19
2	5	6	40	7	4	90	45	60	12	14	75	8							

The super cool thing about binary heaps

Instead of storing it as an actual tree with arrows, because the shape is determined solely by the number of nodes...



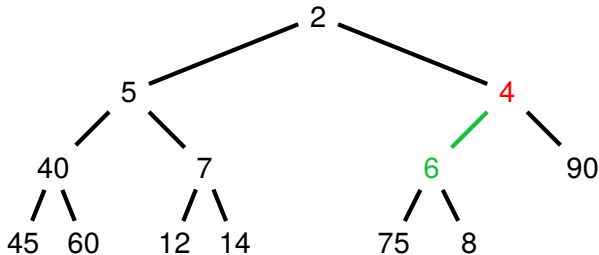
...we can store a binary heap in an array!

In order by level, from left to right within each level:

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19
2	5	6	40	7	4	90	45	60	12	14	75	8							

The super cool thing about binary heaps

Instead of storing it as an actual tree with arrows, because the shape is determined solely by the number of nodes...



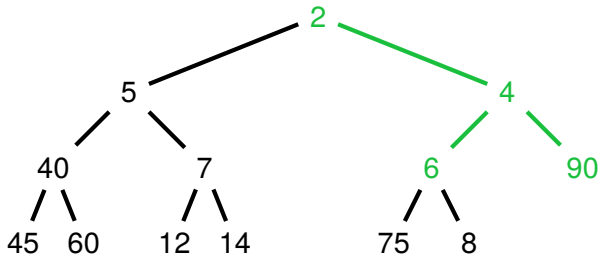
...we can store a binary heap in an array!

In order by level, from left to right within each level:

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19
2	5	4	40	7	6	90	45	60	12	14	75	8							

The super cool thing about binary heaps

Instead of storing it as an actual tree with arrows, because the shape is determined solely by the number of nodes...



...we can store a binary heap in an array!

In order by level, from left to right within each level:

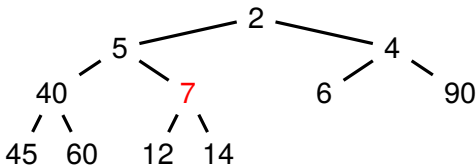
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19
2	5	4	40	7	6	90	45	60	12	14	75	8							

Finding parents and children

Because the structure is *implicit*, we can't just follow arrows

Suppose i is the index of a node. E.g. 7, at index 4.

How can we find its...



0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23
2	5	4	40	7	6	90	45	60	12	14	75	8											

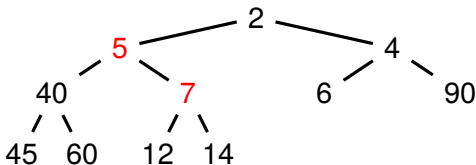
Finding parents and children

Because the structure is *implicit*, we can't just follow arrows

Suppose i is the index of a node. E.g. 7, at index 4.

How can we find its...

$$\text{parent} \quad \lfloor (i - 1)/2 \rfloor \quad \lfloor (4 - 1)/2 \rfloor = 1 \quad \text{vec}[1] == 5$$



0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23
2	5	4	40	7	6	90	45	60	12	14	75	8											

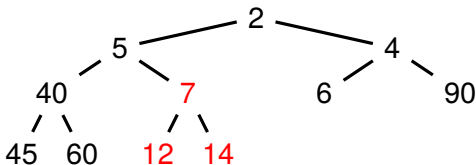
Finding parents and children

Because the structure is *implicit*, we can't just follow arrows

Suppose i is the index of a node. E.g. 7, at index 4.

How can we find its...

parent	$\lfloor (i-1)/2 \rfloor$	$\lfloor (4-1)/2 \rfloor = 1$	$\text{vec}[1] == 5$
left child	$2 \times i + 1$	$2 \times 4 + 1 = 9$	$\text{vec}[9] == 12$
right child	$2 \times i + 2$	$2 \times 4 + 2 = 10$	$\text{vec}[10] == 14$



0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23
2	5	4	40	7	6	90	45	60	12	14	75	8											

Priority queues in practice

Priority queues have many applications:

- Modeling any to-do list where some items are more urgent than others (duh)
- Scheduling (operating systems, simulation, etc.)
- A* search
 - ▶ Version of graph search
 - ▶ Uses a *distance heuristic* as priority
 - ▶ **Likely** to find short paths,
 - ▶ But not guaranteed to find shortest
- Sorting, with heapsort!
 - ▶ Insert vector elements in a binary heap ($\mathcal{O}(\log n)$ each)
 - ▶ Take them out *in sorted order*! ($\mathcal{O}(\log n)$ each)
 - ▶ **Total:** $\mathcal{O}(n \log n)$
 - ▶ Part of your job for homework 5.
- Dijkstra's algorithm (soon!)

More examples

- <https://www.cs.usfca.edu/~galles/visualization/Heap.html>

Next time: Data design